ChatGPT

Design Ops Workspace PRD for a 50-person Design Organisation

Executive summary

This PRD proposes a lightweight, scalable Design Ops app (≈50 users) using **IA Option A**: **role-based Home + universal object model**. The product sits between flexible database tools and workflow trackers, but is explicitly tuned for design organisations operating across Figma, engineering repos/PRs (often surfaced in Visual Studio workflows), and "Vibe coding" projects.

At a glance, the product is a **single source of operational truth** for design delivery, critique health, and impact storytelling—organised by **seven pillars/domains**, owned by **Domain Design Heads**, and fully roll-upable for the **Head of Design/CDO** via reliable metrics (throughput, velocity, review latency, blocked work, and impact slide aggregation).

A crucial design choice is to treat leadership reporting as a consequence of day-to-day design value (My Work, Critique Queue, and simple evidence capture): dashboards must compute from the same objects designers already touch. This aligns with Design Ops as orchestration/optimisation of people + process + craft rather than a "tracking tool". [1]

The PRD recommends a hybrid architecture:

- **Universal core tables** and typed fields for consistent roll-ups
- **Extensible custom fields** for per-pillar variation without breaking metrics
- **Event/audit trail** to compute latency, throughput, and trends reliably
- **RLS enforced at the database layer** (defence in depth) for multi-pillar visibility boundaries and leadership roll-ups [2]
- **Integrations that are event-first** (webhooks/service hooks/events APIs), backed by scheduled **reconciliation** for correctness and missed events [3]
- Optional AI via **RAG** (retrieval-augmented generation) to draft summaries and impact slides credibly with provenance [4]

Assumptions and scope boundaries

Assumptions to proceed without clarification

- Authentication method is **unspecified**; we assume email + SSO upgrade path is required (leadership adoption often pushes SSO). [5]
- Migration/import from Airtable/Monday is **unspecified**; we assume one-time import is MVP-critical, while continuous sync is optional. Relevant API rate limits and cost models must be respected. [6]
- Two pillar names are unknown; pillars are treated as **configurable records** (seeded list editable by admins).
- "Vibe projects" are unspecified; we treat them as a **generic code provider adapter**, initially link-based, later event-synced if/when APIs are known.

Non-goals for the first mature release

- Replace Jira as the system of record for engineering delivery (we integrate; we do not compete).
- Full Airtable-level formula engine or full Monday automation marketplace.
- Deep Figma file parsing at scale (start with links + webhooks; richer ingestion later).
- Real-time multi-user collaborative grid editing (nice-to-have, not required at 50 seats).

Design Ops framing

This product is positioned as a Design Ops layer: orchestrating and optimising people, process, and craft to amplify design's value/impact at scale. [1]

Vision and positioning

Product vision

A **Design Operations Workspace** that:

- Gives designers a daily home for "what matters now" (work, blockers, reviews)
- Makes critique and review throughput **visible and manageable**
- Produces leadership roll-ups and weekly narratives **without manual decks**
- Connects design work to implementation signals (PRs, repos, "Vibe projects")
- Builds a trustworthy "design memory" (impact evidence + AI summaries)

Positioning statement

For a multi-pillar design organisation delivering alongside engineering, this product is a lightweight operational backbone that combines database flexibility with workflow clarity—optimised for design critique cycles and outcome storytelling.

Concept trade-offs

Positioning emphasis	What it optimises	What you sacrifice	Fit for your org
Critique-first	Review latency, quality loops, fewer "stuck in review" items	Less immediate focus on impact storytelling	Very strong (explicit Critique Queue requirement)
Impact-first	Leadership confidence, success story aggregation	Risk of "reporting tax" without good capture UX	Strong (explicit Impact Slides roll-up requirement)
Code+Design delivery	Fewer drift handoffs; engineers trust "ready" state	Risk of becoming an engineering tracker	Strong (explicit code-focused views)
Balanced OS (recommended)	Adoption + reporting from shared objects	Requires disciplined schema design	Best overall for 7-pillar roll-ups

Recommendation: **Balanced OS** as the base, with **Critique Queue** as the daily differentiator and **Impact Slides** as low-friction evidence capture.

Personas and journeys

This section is written in a “Lenny-style” operational tone (clear jobs-to-be-done, journeys, acceptance criteria, success metrics). It’s intended to be implementable, not aspirational.

Designer persona

Who they are
IC designers shipping work across one or more pillars, collaborating with reviewers and engineers, living primarily in Figma plus communication tools.

Primary jobs-to-be-done

- Plan and track design items without creating admin overhead
- Get timely reviews, capture decisions, and unblock work
- Provide handoff-ready context to engineering without duplicating effort
- Capture impact evidence when it’s easy (not as a quarterly scramble)

Key journey

1. **Start the day in My Work**
   Sees active items, due dates, and “waiting on review” states.
2. **Request critique from within a Design Item**
   Assigns reviewers or chooses a pillar review pool; adds a Figma link.
3. **Triages Critique Queue**
   Responds to feedback and marks critique threads as addressed/resolved.
4. **Marks readiness for dev**
   Updates status to “Ready for Dev” / “Approved”; attaches engineering context.
5. **Links code signals**
   Adds PR/repo links or “Vibe project” link (or system auto-suggests from integration).
6. **Adds an Impact Slide (optional but encouraged)**
   Uses an “Impact Slide draft” template; attaches evidence (before/after, metrics, quote).

Acceptance criteria (selected)

- Given I am a designer, when I open Home, then I see **My Work**, **My Review Requests**, and **My Blockers** within one screen.
- Given a design item has a Figma link, when I request critique, then the system creates a **Critique thread** and puts it into the reviewers’ **Critique Queue**.
- Given I receive feedback, when I mark a critique as “Addressed”, then the item’s review status updates and the reviewer is notified (in-app; Slack optional).
- Given an item is blocked, when I set “Blocked reason” and “Blocked since”, then it appears in blocked roll-ups for my pillar and CDO.

Designer success metrics

- Median **review latency to first response** decreases (target defined per pillar).
- % of items “waiting for review” older than SLA decreases.
- % of shipped/design-complete items linked to at least one evidence artefact (Impact Slide or decision log) increases.

Domain Design Head persona (pillar owner)

Who they are
Leads a pillar (e.g., Banking & Lending). Responsible for resourcing, workflow health, and quality. Needs weekly reporting with minimal manual effort.

Primary jobs-to-be-done

- Monitor throughput and bottlenecks within the pillar
- Keep review load healthy (reviewers not overloaded; SLAs met)
- Identify blocked work early and intervene
- Curate a rolling set of impact evidence and success stories

Key journey

1. Opens **Pillar Home** (default landing)
   Sees WIP, blocked ageing, review queue SLA, throughput trend.
2. Drills into **Project dashboards**
   Finds "at risk" projects or items in prolonged review.
3. Manages **review pool**
   Adjusts rotation, assigns reviewers, and sets review SLA expectations.
4. Curates **Impact Slides**
   Approves, tags, and groups slides into "Success Stories" collections.
5. Prepares weekly summary for CDO
   Exports narrative (or AI-drafts) from pillar metrics + top evidence.

Acceptance criteria

- Given I am a Domain Design Head, when I open my dashboard, then I can see my pillar's **throughput**, **review latency**, **blocked ageing**, and **top impact slides** for a chosen time window.
- Given an item exceeds review SLA, when the SLA threshold is crossed, then it appears in a "Needs Attention" list with responsible reviewer/owner.
- Given a success story collection is created, when I add slides to it, then it is visible in the CDO roll-up.

Domain Head success metrics

- Pillar-level median review latency reduced and stable.
- Blocked rate (snapshot + time-weighted) trending down.
- Increase in "impact evidence coverage" (evidence-linked work).

Head of Design/CDO persona

Who they are
Leads all domain heads; needs roll-up reporting and a defensible narrative about design's output and impact.

Primary jobs-to-be-done

- Track delivery health across seven pillars (comparative)
- Identify hotspots (where velocity/throughput is falling, or reviews are stuck)
- Produce leadership narratives and success stories consistently

- Maintain trust in metrics (no gaming, no missing data)

Key journey

1. Opens **CDO Home**
   Sees pillar comparison and “red/yellow/green” operational health.
2. Drills to pillar and project level
   Understands root causes (blocked reasons, reviewer bottlenecks).
3. Reviews **Impact roll-up**
   Scans success story gallery; selects top stories.
4. Exports a weekly/monthly narrative
   Generates digest (manual or AI-assisted) with links to evidence.

Acceptance criteria

- Given I am the CDO, when I open the roll-up, then I can compare pillars on throughput, velocity, review latency, blocked rate, and evidence coverage.
- Given I choose a time window, when metrics are computed, then definitions are consistent across pillars (standard status taxonomy and event semantics).
- Given I export a narrative, when it is generated, then every claim links to underlying evidence (items, critiques, impact slides).

CDO success metrics

- Weekly reporting time reduced (hours → minutes).
- Confidence and usage of dashboards increases (weekly active views).
- Leadership “success story” cadence becomes predictable (e.g., monthly pack generated from evidence).

Information architecture and navigation

IA Option A: role-based Home + universal object model.

Navigation principles

- **Role-based default landing**
  Designers land on “My Work + Critique Queue”; Domain Heads on “Pillar Dashboard”; CDO on “Org Roll-up”.
- **Universal objects** (Design Items, Projects, Critiques, Impact Slides) remain consistent across roles.
- **Saved views** are discoverable but not the first interaction (avoid “blank base” problem).
- **Work starts from Home, not from configuration**.

Mermaid IA diagram

```
flowchart TB
  H[Home] --> D[Designer Home<br/>My Work + Critique Queue]
  H --> P[Pillar Home<br/>Domain dashboard]
  H --> O[CDO Home<br/>Org roll-up]

  H --> Items[Design Items]
```

```
  H --> Proj[Projects]
  H --> Crit[Critiques]
  H --> Impact[Impact Slides]
  H --> Views[Views Library]
  H --> Integrations[Integrations]
  H --> Settings[Settings/Admin]

  Views --> Grid[Table/Grid]
  Views --> Kanban[Board/Kanban]
  Views --> Time[Timeline]
  Views --> CQ[Critique Queue]
  Views --> IG[Impact Gallery]
  Views --> Code[Code-focused View]
  Views --> Eng[Read-only Engineering View]
```

Data model and security

This section focuses on “PRD-ready” schema clarity: tables, indexes, RLS patterns, and impact-slide roll-ups.

Data model overview

Core design: **typed core fields + optional custom fields**, plus an **event trail** for accurate metrics.

- Typed core fields support consistent roll-ups across pillars.
- Custom fields support pillar differences without breaking reporting.
- Event trail powers latency and throughput trend analysis.

Sample SQL-like schema

Below is an illustrative Postgres schema (suitable for Supabase or self-hosted Postgres). It includes a minimum viable set for your required entities and views.

> RLS is enabled where user-accessed tables exist; policies below illustrate role + pillar membership + item-level sharing. Postgres RLS and policy mechanics are documented in the official Postgres docs. [7]
> Supabase’s RLS guidance and helper functions (e.g., `auth.uid()`, `anon` / `authenticated` roles, and “policies as implicit WHERE clauses”) are documented in Supabase docs. [8]

```sql
-- Core enums (keep as TEXT + CHECK if you want easier evolution)
-- For PRD illustration, we use CHECK constraints for flexibility.

create table orgs (
  id uuid primary key,
  name text not null,
  created_at timestamptz not null default now()
);
```

```sql
create table pillars (
  id uuid primary key,
  org_id uuid not null references orgs(id),
  name text not null,
  domain_head_user_id uuid null,
  created_at timestamptz not null default now(),
  unique (org_id, name)
);

create table profiles (
  user_id uuid primary key,
  org_id uuid not null references orgs(id),
  email text not null,
  display_name text not null,
  role text not null check (role in (
    'designer','reviewer','domain_head','cdo','engineer','pm','admin'
  )),
  is_active boolean not null default true,
  created_at timestamptz not null default now(),
  unique (org_id, email)
);

create table pillar_memberships (
  org_id uuid not null references orgs(id),
  pillar_id uuid not null references pillars(id),
  user_id uuid not null references profiles(user_id),
  membership_type text not null check (membership_type in
('member','lead','reviewer_pool')),
  created_at timestamptz not null default now(),
  primary key (org_id, pillar_id, user_id)
);

create table projects (
  id uuid primary key,
  org_id uuid not null references orgs(id),
  pillar_id uuid not null references pillars(id),
  name text not null,
  description text null,
  status text not null check (status in
('planned','active','paused','done','archived')),
  start_date date null,
  end_date date null,
  created_by uuid not null references profiles(user_id),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create index projects_org_pillar_status_idx on projects (org_id, pillar_id,
status);
create index projects_org_end_date_idx on projects (org_id, end_date);
```

```
create table project_members (
  project_id uuid not null references projects(id),
  user_id uuid not null references profiles(user_id),
  access_level text not null check (access_level in
('owner','editor','viewer')),
  created_at timestamptz not null default now(),
  primary key (project_id, user_id)
);

create table design_items (
  id uuid primary key,
  org_id uuid not null references orgs(id),
  pillar_id uuid not null references pillars(id),
  project_id uuid null references projects(id),
  title text not null,
  description text null,
  item_type text not null check (item_type in
('feature','flow','experiment','bug','research','other')),
  status text not null check (status in (

'backlog','in_progress','in_review','approved','ready_for_dev','done','archived'
  )),
  priority text not null check (priority in ('p0','p1','p2','p3')),
  owner_id uuid not null references profiles(user_id),
  due_date date null,
  effort_points int null check (effort_points is null or effort_points
between 1 and 13),
  is_blocked boolean not null default false,
  blocked_reason text null,
  blocked_since timestamptz null,
  created_by uuid not null references profiles(user_id),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create index design_items_org_pillar_status_idx on design_items (org_id,
pillar_id, status);
create index design_items_org_owner_status_idx on design_items (org_id,
owner_id, status);
create index design_items_org_project_status_idx on design_items (org_id,
project_id, status);
create index design_items_org_updated_idx on design_items (org_id,
updated_at);
create index design_items_org_blocked_idx on design_items (org_id,
is_blocked, blocked_since);

-- Item-level sharing (for cross-pillar work, or wider visibility)
create table design_item_shares (
  design_item_id uuid not null references design_items(id),
  shared_with_user_id uuid not null references profiles(user_id),
  access_level text not null check (access_level in
```

```
('viewer','commenter','editor')),
  created_by uuid not null references profiles(user_id),
  created_at timestamptz not null default now(),
  primary key (design_item_id, shared_with_user_id)
);

-- Event trail for accurate metrics
create table design_item_events (
  id uuid primary key,
  design_item_id uuid not null references design_items(id),
  org_id uuid not null references orgs(id),
  event_type text not null check (event_type in (
    'created','status_changed','blocked','unblocked','owner_changed',
    'review_requested','approved','done','commented'
  )),
  from_status text null,
  to_status text null,
  actor_user_id uuid not null references profiles(user_id),
  occurred_at timestamptz not null default now(),
  metadata jsonb null
);

create index design_item_events_item_time_idx on design_item_events
(design_item_id, occurred_at);
create index design_item_events_org_type_time_idx on design_item_events
(org_id, event_type, occurred_at);

-- Critique threads
create table critiques (
  id uuid primary key,
  org_id uuid not null references orgs(id),
  design_item_id uuid not null references design_items(id),
  requested_by uuid not null references profiles(user_id),
  reviewer_id uuid not null references profiles(user_id),
  state text not null check (state in
('pending','in_progress','addressed','approved','closed')),
  requested_at timestamptz not null default now(),
  first_response_at timestamptz null,
  resolved_at timestamptz null,
  created_at timestamptz not null default now()
);

create index critiques_item_state_idx on critiques (design_item_id, state);
create index critiques_reviewer_state_idx on critiques (reviewer_id, state);
create index critiques_org_requested_at_idx on critiques (org_id,
requested_at);

create table critique_comments (
  id uuid primary key,
  critique_id uuid not null references critiques(id),
  org_id uuid not null references orgs(id),
```

```
  author_id uuid not null references profiles(user_id),
  body text not null,
  created_at timestamptz not null default now(),
  resolved_at timestamptz null
);

create index critique_comments_crit_time_idx on critique_comments
(critique_id, created_at);

-- Links: Figma + code
create table figma_links (
  id uuid primary key,
  org_id uuid not null references orgs(id),
  design_item_id uuid not null references design_items(id),
  url text not null,
  file_key text null,
  node_id text null,
  created_by uuid not null references profiles(user_id),
  created_at timestamptz not null default now()
);

create index figma_links_item_idx on figma_links (design_item_id);
create index figma_links_file_key_idx on figma_links (file_key);

create table code_links (
  id uuid primary key,
  org_id uuid not null references orgs(id),
  design_item_id uuid not null references design_items(id),
  provider text not null check (provider in
('github','gitlab','azure_devops','vibe','other')),
  repo text null,
  pr_number int null,
  commit_sha text null,
  url text not null,
  status text null,
  last_synced_at timestamptz null,
  created_by uuid not null references profiles(user_id),
  created_at timestamptz not null default now()
);

create index code_links_item_idx on code_links (design_item_id);
create index code_links_provider_repo_pr_idx on code_links (provider, repo,
pr_number);

-- Impact slides and roll-ups
create table impact_slides (
  id uuid primary key,
  org_id uuid not null references orgs(id),
  pillar_id uuid not null references pillars(id),
  project_id uuid null references projects(id),
  title text not null,
```

```
  summary text null,
  outcome_type text not null check (outcome_type in (

'conversion','engagement','retention','cost','risk','quality','csat','delivery','other'
  )),
  asset_url text not null,
  thumb_url text null,
  created_by uuid not null references profiles(user_id),
  created_at timestamptz not null default now(),
  approved_by uuid null references profiles(user_id),
  approved_at timestamptz null,
  tags text[] null
);

create index impact_slides_org_pillar_time_idx on impact_slides (org_id,
pillar_id, created_at);
create index impact_slides_org_project_time_idx on impact_slides (org_id,
project_id, created_at);

create table impact_slide_links (
  impact_slide_id uuid not null references impact_slides(id),
  design_item_id uuid not null references design_items(id),
  created_at timestamptz not null default now(),
  primary key (impact_slide_id, design_item_id)
);

create index impact_slide_links_item_idx on impact_slide_links
(design_item_id);

-- Success stories (roll-up collections)
create table success_stories (
  id uuid primary key,
  org_id uuid not null references orgs(id),
  pillar_id uuid null references pillars(id),
  title text not null,
  narrative text null,
  created_by uuid not null references profiles(user_id),
  created_at timestamptz not null default now()
);

create table success_story_slides (
  success_story_id uuid not null references success_stories(id),
  impact_slide_id uuid not null references impact_slides(id),
  primary key (success_story_id, impact_slide_id)
);

-- Saved views
create table views (
  id uuid primary key,
  org_id uuid not null references orgs(id),
  owner_id uuid not null references profiles(user_id),
```

```
  name text not null,
  scope_type text not null check (scope_type in
('org','pillar','project','personal')),
  scope_id uuid null,
  definition jsonb not null,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create index views_org_scope_idx on views (org_id, scope_type, scope_id);
create index views_owner_idx on views (owner_id);

create table view_shares (
  view_id uuid not null references views(id),
  shared_with_role text null,
  shared_with_pillar_id uuid null references pillars(id),
  shared_with_user_id uuid null references profiles(user_id),
  access_level text not null check (access_level in ('viewer','editor')),
  primary key (view_id, shared_with_role, shared_with_pillar_id,
shared_with_user_id)
);

-- Optional AI memory
create table ai_memory (
  id uuid primary key,
  org_id uuid not null references orgs(id),
  scope_type text not null check (scope_type in
('design_item','project','pillar','org','person')),
  scope_id uuid not null,
  content text not null,
  source_refs jsonb null,
  created_by uuid not null references profiles(user_id),
  created_at timestamptz not null default now()
);

create index ai_memory_scope_time_idx on ai_memory (org_id, scope_type,
scope_id, created_at);
```

RLS policy examples

Policy philosophy

- Default: a user can access objects in pillars they belong to.
- Required exceptions:
- CDO sees all pillars.
- Item-level sharing can extend visibility cross-pillar.
- Project membership can extend visibility cross-pillar.
- Engineering view is read-only: enforce at API/UX, and optionally via RLS write policies.

Postgres RLS requires enabling RLS and creating policies (`CREATE POLICY`, `ALTER TABLE … ENABLE ROW LEVEL SECURITY`). [7]

Supabase documents policies as implicit WHERE clauses and provides `auth.uid()` helpers plus role mapping (`anon`, `authenticated`). [8]

```
-- Enable RLS
alter table profiles enable row level security;
alter table projects enable row level security;
alter table design_items enable row level security;
alter table critiques enable row level security;
alter table impact_slides enable row level security;

-- Helper view: current user's role (implementation differs if not using
Supabase)
-- In Supabase, auth.uid() is available. [8]

-- Profiles: users can view their own profile; CDO/admin can view all in org.
create policy "profiles_self_or_admin"
on profiles for select
using (
  user_id = auth.uid()
  or exists (
    select 1 from profiles p
    where p.user_id = auth.uid() and p.org_id = profiles.org_id
      and p.role in ('cdo','admin')
  )
);

-- Design items: visible if
-- 1) same org and pillar member OR
-- 2) shared explicitly OR
-- 3) in a project where user is a member OR
-- 4) user is CDO/admin
create policy "design_items_visibility"
on design_items for select
using (
  org_id = (select org_id from profiles where user_id = auth.uid())
  and (
    exists (
      select 1 from pillar_memberships m
      where m.org_id = design_items.org_id
        and m.pillar_id = design_items.pillar_id
        and m.user_id = auth.uid()
    )
    or exists (
      select 1 from design_item_shares s
      where s.design_item_id = design_items.id
        and s.shared_with_user_id = auth.uid()
    )
    or exists (
      select 1 from project_members pm
      where pm.project_id = design_items.project_id
```

```
        and pm.user_id = auth.uid()
    )
    or exists (
      select 1 from profiles p
      where p.user_id = auth.uid()
        and p.org_id = design_items.org_id
        and p.role in ('cdo','admin')
    )
  )
);

-- Design items: writes restricted to owners/editors, domain heads, admins,
CDO
create policy "design_items_write"
on design_items for update
using (
  exists (
    select 1 from profiles p
    where p.user_id = auth.uid()
      and p.org_id = design_items.org_id
      and (
        p.role in ('cdo','admin')
        or (p.role = 'domain_head' and exists (
          select 1 from pillar_memberships m
          where m.org_id = design_items.org_id
            and m.pillar_id = design_items.pillar_id
            and m.user_id = auth.uid()
            and m.membership_type in ('lead')
        ))
        or design_items.owner_id = auth.uid()
        or exists (
          select 1 from design_item_shares s
          where s.design_item_id = design_items.id
            and s.shared_with_user_id = auth.uid()
            and s.access_level = 'editor'
        )
      )
  )
);
```

Impact Slides schema comparison table

Table	Key fields	Primary usage	Recommended indexes	Not
`impact_slides`	`pillar_id`, `project_id`, `outcome_type`, `asset_url`, `approved_at`, `tags[]`	Gallery, roll-ups, success story evidence	`(org_id,pillar_id,created_at)`, `(org_id,project_id,created_at)`	Mak `ap` opti ena “lea filte
`impact_slide_links`	`impact_slide_id`, `design_item_id`	Evidence mapping from work → slides	`(design_item_id)`, unique composite PK	Pow cov
`success_stories`	`pillar_id?`, `title`, `narrative`	Curated roll-ups and storytelling packs	`(org_id,pillar_id,created_at)`	Pilla cros
`success_story_slides`	`success_story_id`, `impact_slide_id`	Story composition	composite PK	Ena suc pac

Views and UI patterns per role

This section’s goal is not to prescribe pixels but to define **repeatable patterns** that engineers can implement predictably.

Visual suggestions

Designer landing

Designer Home (default)

- **My Work**
- Active items grouped by status
- “Due this week” and “Blocked” slices
- **Critique Queue**
- Requests *to me* and requests *from me*
- SLA ageing badges (e.g., 0–2d, 3–5d, 6+d)
- **Quick actions**
- New Design Item
- Request Critique
- Add Impact Slide

UI pattern: left nav + central list + right preview drawer.

Domain Design Head landing

Pillar Home

- KPI row: throughput, velocity, review latency, blocked rate
- Trend row: throughput over time; review latency distribution
- Operational panels:
- Blocked ageing list with top reasons
- Review queue SLA breaches (by reviewer or project)
- Evidence feed (new/approved impact slides)

CDO landing

Org Roll-up

- Pillar comparison view:
- stacked throughput bars by pillar (weekly)
- review latency median + P75 by pillar
- blocked snapshot rate by pillar
- “Hot spots” list:
- pillars with rising WIP or review backlogs
- “Success stories” gallery:
- top approved slides by pillar
- curated story collections

Required views mapped to UI patterns

Table/Grid (Airtable-style) - Editable cells, column types (status, select, multi-select, user, date, link, number) - Saved views for role/pillar/project - Inline filter/sort/group + “save view” workflow

Board/Kanban (Monday-style) - Columns by status taxonomy - Drag-and-drop with event logging (“status_changed” events) - WIP limit indicators (optional for Phase 2)

Timeline - Per-project - Start/end dates, milestones (“impact story due”), major critique dates - Dependencies only if needed (avoid heavy PM tooling)

Critique Queue - Inbox semantics: - Assigned to me - Mentioned me (future integration with Slack/ Figma comments) - Review pool items (rotation for the pillar) - States: pending → in progress → addressed → approved/closed - Latency timers: - time since request - time since last response

Impact Slide Gallery - Card grid with filters: - pillar, project, outcome type, approved only - “Evidence coverage” badge per project/pillar (% items with ≥1 slide) - Story builder: - drag slides into a “success story” collection

Code-focused view - Design items filtered to “Ready for Dev”, “In Dev”, “Done” - Shows: - code links grouped by provider and PR status - “handoff checklist” (fields + links) - Intended for design/engineering alignment without rewriting Jira

Read-only engineering view - A constrained saved view: - only key fields, Figma link(s), acceptance notes, related PRs - absolutely no destructive actions - Shareable link for Jira tickets or Slack announcements

Permission model and concrete access rules

Recommended model

Role + pillar membership + item-level sharing (your requested model)

- **Role** determines baseline capability:
- designer/reviewer/engineer/pm: limited writes
- domain_head: pillar-wide visibility + management controls
- cdo/admin: org-wide visibility
- **Pillar membership** determines default visibility
- **Item-level sharing** for cross-pillar or special-case visibility

Rationale: This design lets you keep a single universal workspace (best for cross-domain roll-ups) without forcing separate workspaces per pillar.

Concrete rules (human-readable)

- Designers can:
- view items in their pillar(s)
- edit items they own
- comment/critique on items they're assigned to review
- Reviewers can:
- view items they're reviewing (plus their pillar membership scope)
- update critique states, not arbitrarily change item pillar/owner
- Domain Heads can:
- view all items in their pillar
- assign reviewers, manage critique rotation, approve impact slides
- CDO can:
- view all pillars and all roll-ups
- create org-level saved views and story collections
- Engineers can:
- view read-only engineering views
- see handoff readiness data and links
- not update core Design Item tracking fields unless explicitly granted

Trade-offs

- **Pros**
- Works cleanly for seven pillars with roll-ups
- Minimises operational friction while maintaining confidentiality
- RLS ensures enforcement at the data layer (defence in depth) [2]
- **Cons**
- Requires accurate pillar membership maintenance
- Edge cases: cross-pillar projects need explicit membership/share rules (handled via `project_members` / `design_item_shares`)

Reporting, metrics, and queries

Metric definitions

Metrics should be computed from **events** wherever possible (status transitions, critique timestamps), not only from current state fields—otherwise latency metrics become untrustworthy.

Throughput
Count of design items that hit a terminal status within a time window.

Velocity
Effort-weighted throughput: sum of `effort_points` for items completed in the window.

Review latency
Two explicit metrics: - time-to-first-response: first critique response − review requested - time-to-approval: item approved − review requested

Blocked rate - Snapshot blocked rate: blocked items / active items - Time-weighted blocked rate: blocked duration summed across items / active cycle time

Sample SQL queries

Assume: - `done` = status transitioned to `done` - `review_requested` = event type `review_requested` OR status moved into `in_review`

Throughput (weekly, by pillar)

```
select
  di.pillar_id,
  date_trunc('week', e.occurred_at) as week,
  count(*) as throughput
from design_item_events e
join design_items di on di.id = e.design_item_id
where e.event_type = 'done'
  and e.occurred_at >= now() - interval '12 weeks'
group by 1, 2
order by week desc;
```

Velocity (weekly, by pillar)

```
select
  di.pillar_id,
  date_trunc('week', e.occurred_at) as week,
  sum(coalesce(di.effort_points, 0)) as velocity_points
from design_item_events e
join design_items di on di.id = e.design_item_id
where e.event_type = 'done'
  and e.occurred_at >= now() - interval '12 weeks'
```

```
group by 1, 2
order by week desc;
```

Review latency to first response (median, by pillar)

```
with req as (
  select design_item_id, min(occurred_at) as requested_at
  from design_item_events
  where event_type = 'review_requested'
  group by 1
),
first_resp as (
  select c.design_item_id, min(cc.created_at) as first_response_at
  from critiques c
  join critique_comments cc on cc.critique_id = c.id
  group by 1
)
select
  di.pillar_id,
  percentile_cont(0.5) within group (order by (first_resp.first_response_at
- req.requested_at)) as median_time_to_first_response
from design_items di
join req on req.design_item_id = di.id
join first_resp on first_resp.design_item_id = di.id
where req.requested_at >= now() - interval '12 weeks'
group by 1;
```

Blocked snapshot rate (current, by pillar)

```
with active as (
  select pillar_id,
         count(*) filter (where status in
('in_progress','in_review','approved','ready_for_dev')) as active_items,
         count(*) filter (where is_blocked = true and status in
('in_progress','in_review','approved','ready_for_dev')) as blocked_items
  from design_items
  where status not in ('done','archived')
  group by 1
)
select
  pillar_id,
  blocked_items,
  active_items,
  case when active_items = 0 then 0 else blocked_items::float / active_items
end as blocked_rate_snapshot
from active;
```

Visualisations and where to use them

- **CFD (cumulative flow diagram)**: show WIP by status over time (Domain Head + CDO)
- **Ageing WIP**: histogram or stacked bars of "days in status" (Domain Head)
- **Stacked throughput bars** by pillar per week (CDO)
- **Latency distribution** (median + P75) by pillar and by reviewer group (CDO + Domain Head)

Mermaid metrics pipeline flow

```
flowchart LR
  A[Design Items<br/>current state] --> M[Metrics compute]
  B[Item Events<br/>status transitions] --> M
  C[Critiques + Comments] --> M
  D[Impact Slides + Links] --> M
  E[Code Links] --> M

  M --> V1[Designer dashboards<br/>My Work, Critique Queue]
  M --> V2[Project dashboards]
  M --> V3[Pillar dashboards]
  M --> V4[CDO roll-up]
```

Integration design and AI layer

Integration principles

- Prefer official event systems:
- Figma webhooks, Slack Events API, repo webhooks, Azure DevOps service hooks, Jira webhooks.
- Back webhooks with reconciliation:
- Webhooks can be missed or delayed; reconciliation ensures eventual correctness.
- Verify inbound events:
- Validate signatures/tokens/passcodes before processing.

Figma integration

Auth options - OAuth2 is supported for user-authorised access; Figma documents the OAuth flow and warns that embedded WebViews are not supported. [9] - Personal access tokens exist, but they are powerful; Figma notes they can't be restricted to a subset of files and should only be shared with trusted integrations. [10]

Webhooks - Webhooks attach to contexts (team/project/file) with documented per-context limits and plan-level totals; Figma also recommends exposing a public endpoint for local dev. [11] - Figma retries failed webhook requests with exponential backoff (3 retries at defined intervals). [12] - Webhook security uses a required `passcode` that is echoed back in events for verification. [13]

Events to use - `FILE_UPDATE` triggers "within 30 minutes of editing inactivity" (useful for "recent activity" signals). [14] - `FILE_VERSION_UPDATE` triggers on named version creation (strong "ready" signal). [14] - `FILE_COMMENT` supports critique workflows and mentions. [14] - `LIBRARY_PUBLISH` can drive design system update notifications; payload may split across multiple events for large publishes. [14]

Rate limits - Figma rate limits vary by seat type, endpoint tier, and the plan/location of the resource; Figma provides a tier table and notes updated limits as of 17 Nov 2025. [15]

Design recommendation - MVP: store Links + ingest webhook activity as lightweight event hints. - Phase 2: add "reconciliation job" for canonical metadata; cache aggressively to avoid hitting plan/tier limits. [16]

GitHub / GitLab / Azure DevOps integration

Because engineering users are "VS/Vibe" oriented, the first value is linking design items to PRs and review status.

GitHub - Webhooks send events with headers including `X-Hub-Signature-256` when configured with a secret; documentation recommends using SHA-256 signature header over SHA-1. [17] - GitHub explicitly recommends validating webhook signatures before processing to avoid tampering and wasted compute. [18] - REST endpoints exist for pull request operations (list, create, merge, reviews). [19]

GitLab - Webhooks support a secret token sent in `X-Gitlab-Token`. [20] - Merge request APIs are available to retrieve status and review details. [21]

Azure DevOps - Service hooks allow event subscriptions that send JSON to a public endpoint; Microsoft describes publishers/events/subscriptions/consumers. [22] - REST APIs exist for pull request operations and related threads/reviewers (useful for "review requested/approved" style signals). [23]

Vibe as generic adapter

Treat "Vibe" as:

- `provider = vibe`
- Minimal schema: `url`, optional `project_id` and `status`
- Optional future: webhook receiver + mapping logic if/when Vibe exposes events

Slack integration

- Incoming webhooks are a straightforward outbound notification mechanism (post a JSON payload to a unique URL). [24]
- Slack Events API is the inbound event mechanism, delivered either via HTTP endpoint or Socket Mode; Slack emphasises event subscriptions and acknowledges retry behaviour. [25]

MVP recommendation: - Phase 1: outbound notifications via incoming webhooks (low complexity). - Phase 2+: inbound Events API for interactive actions ("/designops review", approve slide, etc.).

Jira integration

- Jira Cloud webhooks support filtering via JQL for subsets of webhook events; Jira documents restrictions, concurrency limits, and expiration/refresh requirements for dynamically registered webhooks. [26]
- Jira Cloud webhook deliveries can be secured using a secret which generates an HMAC signature in `X-Hub-Signature`; Jira documents how to validate it. [27]
- Jira Cloud also notes OAuth2 app webhooks can be secured with bearer authentication in the `Authorization` header signed with the client secret (verify JWT). [28]

- Jira platform rate limiting is documented and must be handled (429/backoff). [29]

Airtable / Monday import patterns

- Airtable documents rate limits as “5 requests per second per base” and additional token traffic limits. [30]
- Monday API rate limits are based on GraphQL complexity points with documented budgets. [31]

Import guidance: - Implement a mapping UI: status, owner, pillar, project, dates, links. - Batch and throttle per documented limits; store “source record ids” for trace-back. - Phase 1 supports CSV import + optional API import; continuous sync can be deferred.

Optional AI layer

Why RAG RAG combines parametric generation with a retriever over an external memory store; it improves factuality and supports provenance more than “model-only memory”. [4]

What to index - Design items: title, description, status history, blocked reasons - Critique comments (careful: sensitive) - Impact slides (summary + tags + outcomes) - Integration events (PR titles, merged dates, etc.)

Vector store options (practical for 50 users) - Postgres with vector extension (fits a single-stack approach; especially convenient if using Supabase where extensions like pgvector are available in the managed ecosystem). [32] - Dedicated vector DB is optional; at this scale, operational simplicity often wins.

Prompt templates

Impact slide drafting:

```
System: You are a DesignOps assistant. Produce concise, evidence-linked
impact slide drafts.
User: Draft an Impact Slide for the following Design Item and evidence. Use
UK English.
Inputs:
- Design Item: {{title}}, {{summary}}, {{pillar}}, {{project}},
{{status_timeline}}
- Evidence snippets: {{evidence_chunks_with_links}}
Constraints:
- Output: title, 2–3 bullet summary lines, outcome type guess, evidence
links, confidence notes.
- Do not invent metrics; if none exist, propose placeholders clearly marked
as TBD.
```

CDO weekly narrative:

```
System: You summarise operational metrics and success stories for design
leadership.
User: Generate a weekly roll-up narrative for the CDO.
```

```
Inputs:
- Time window: {{start}} to {{end}}
- Pillar metrics table: {{throughput, velocity, review_latency,
blocked_rate}}
- Top impact slides per pillar (approved): {{slides_with_links}}
Constraints:
- Highlight hotspots and explain probable causes using only retrieved
evidence.
- End with 3 recommended actions and owners (Domain Head, reviewer pool, or
design lead).
```

MVP roadmap and implementation trade-offs

Platform trade-offs: Supabase vs Firebase vs self-hosted Postgres

Option	Pros	Cons	Security model	Best fit
Supabase [33]	Postgres + RLS, strong for relational roll-ups; good local dev; storage integrates with RLS	You still own schema/RLS complexity	Postgres RLS + policies; Supabase documents RLS patterns and helper functions	Recommended for this PRD [34]
Firebase (Firestore) via Google [35]	Very fast to prototype; managed scaling; realtime easy	Harder relational analytics and cross-entity roll-ups; complex “joins” for dashboards	Firestore security rules are powerful but different; documented propagation delays and rules syntax	If you prioritise speed over analytics [36]
Self-hosted Postgres	Maximum control, simplest mental model for analytics	Operational burden (backups, migrations, auth, storage)	Native Postgres RLS supported	If you already have strong infra ops [7]

Recommended stack for VS Code development

- Frontend: React + Vite (or Next.js if you prefer integrated API routes)
- UI: system tokens + minimal grayscale components
- Backend: Supabase (Postgres + RLS) + edge functions for webhooks
- Background jobs: scheduled functions (cron) for reconciliation and metric aggregation
- Observability: structured logs for webhooks + metric job runs

Phase roadmap (deliverables, effort, roles)

Phase	Duration (weeks)	Primary deliverables	Team roles	Key risks + mitigations
Phase One	4–6	Universal objects + IA Option A Home; Table/Grid + Kanban; Critique Queue; read-only engineering view; pillar membership + RLS; basic import (CSV)	1 FE, 1 BE, 0.5 product/ design	Risk: low adoption → mitigate by making Critique Queue and My Work excellent
Phase Two	4–6	Pillar dashboards + CDO roll-up; event trail reliability; Impact Slide gallery + linking; Slack outbound notifications	1 FE, 1 BE, 0.25 data/ analytics	Risk: metric mistrust → mitigate with event definitions + audit trail UX
Phase Three	6–10	Integrations maturity: Figma webhooks + reconciliation; repo adapters (GitHub/GitLab/Azure DevOps); Jira linking; optional AI RAG summaries and slide drafting	1 FE, 1 BE, 0.5 integrations	Risk: webhook security/reliability → mitigate with signature verification and recon jobs [37]

Integration sources to prioritise (official)

At build time, prioritise official API docs for:

- Figma [38] REST API, OAuth, webhooks, rate limits, webhook security. [39]
- GitHub [40] webhooks + signature validation + pull request APIs. [41]
- GitLab [42] webhooks + merge request APIs. [43]
- Microsoft [44] Azure DevOps service hooks + PR APIs. [45]
- Slack [46] incoming webhooks + Events API. [47]
- Atlassian [48] Jira Cloud webhooks + security + rate limits. [49]
- Airtable [50] Web API + rate limits for import throttling. [51]
- monday.com [52] API rate limits/complexity model for import. [53]

Key risks and mitigations

- **Data quality for metrics**
  Mitigation: event logging on every status change; clear status taxonomy; simple UX nudges (e.g., “blocked since required”).
- **Permission edge cases (cross-pillar work)**
  Mitigation: explicit sharing tables + project membership; write permission limited by role; enforce with RLS. [2]
- **Webhook authenticity and delivery issues**
  Mitigation: verify signatures/tokens/passcodes; use retries/backoff; reconciliation jobs. [54]
- **Perceived “reporting tax” from Impact Slides**
  Mitigation: “draft from item” templates; optional AI drafting with strict “no invented metrics”; approval workflow generates leadership pack automatically. [4]

1 DesignOps 101
https://www.nngroup.com/articles/design-operations-101/?utm_source=chatgpt.com

2 7 35 52 Documentation: 18: 5.9. Row Security Policies
https://www.postgresql.org/docs/current/ddl-rowsecurity.html?utm_source=chatgpt.com

3 14 33 Events | Developer Docs
https://developers.figma.com/docs/rest-api/webhooks-events/

4 Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
https://arxiv.org/abs/2005.11401?utm_source=chatgpt.com

5 Auth | Supabase Docs
https://supabase.com/docs/guides/auth?utm_source=chatgpt.com

6 30 51 Rate limits - Airtable Web API
https://www.airtable.com/developers/web/api/rate-limits?utm_source=chatgpt.com

8 32 34 Row Level Security | Supabase Docs
https://supabase.com/docs/guides/database/postgres/row-level-security

9 39 48 Authentication | Developer Docs
https://developers.figma.com/docs/rest-api/authentication/?utm_source=chatgpt.com

10 Manage personal access tokens
https://help.figma.com/hc/en-us/articles/8085703771159-Manage-personal-access-tokens?utm_source=chatgpt.com

11 12 Webhooks V2 | Developer Docs
https://developers.figma.com/docs/rest-api/webhooks/

13 37 40 54 Security | Developer Docs
https://developers.figma.com/docs/rest-api/webhooks-security/

15 16 Rate Limits | Developer Docs
https://developers.figma.com/docs/rest-api/rate-limits/

17 41 Webhook events and payloads - GitHub Docs
https://docs.github.com/en/webhooks/webhook-events-and-payloads

18 Validating webhook deliveries - GitHub Docs
https://docs.github.com/en/webhooks/using-webhooks/validating-webhook-deliveries

19 REST API endpoints for pull requests
https://docs.github.com/en/rest/pulls?utm_source=chatgpt.com

20 42 43 Webhooks | GitLab Docs
https://docs.gitlab.com/user/project/integrations/webhooks/

21 Merge requests API
https://docs.gitlab.com/api/merge_requests/?utm_source=chatgpt.com

22 45 Integrate with Service Hooks - Azure DevOps | Microsoft Learn
https://learn.microsoft.com/en-us/azure/devops/service-hooks/overview?view=azure-devops

23 Pull Requests - REST API (Azure DevOps Git)
https://learn.microsoft.com/en-us/rest/api/azure/devops/git/pull-requests?view=azure-devops-rest-7.1&utm_source=chatgpt.com

24 38 47 Sending messages using incoming webhooks | Slack Developer Docs
https://docs.slack.dev/messaging/sending-messages-using-incoming-webhooks

25 The Events API | Slack Developer Docs
https://docs.slack.dev/apis/events-api/

26 27 28 49 50 Webhooks
https://developer.atlassian.com/cloud/jira/platform/webhooks/

29 Rate limiting - Jira Cloud platform
https://developer.atlassian.com/cloud/jira/platform/rate-limiting/?utm_source=chatgpt.com

31 44 53 Rate limits - Apps Framework - Monday.com
https://developer.monday.com/api-reference/docs/rate-limits?utm_source=chatgpt.com

36 46 Get started with Cloud Firestore Security Rules - Firebase
https://firebase.google.com/docs/firestore/security/get-started?utm_source=chatgpt.com